

Reviewer Pack — Phase Merging Complexity Bound

Entry 3b94aaddf6cd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 05:58:57 UTC

Phase Merging Complexity Bound

Entry ID: 3b94aaddf6cd **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 05:58:57 UTC

1. Conjecture

Field A (mathematical branch): Tropical Circuit Weight Analysis (TCWA) in BOUNDED_ARITHMETIC

Field B (complexity object): Minkowski Sum Complexity

Statement:

The tropical proof rank of a phase space obtained via Phase Merging of two circuits is at most the sum of their individual tropical proof ranks.

Rationale (proposer's reasoning):

Tests A1 (Phase Stability Axiom) by verifying if merged phase spaces' complexity scales additively with individual ranks, ensuring bounded arithmetic proof strength remains decomposable under superposition.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time:)

Framework membership: framework fw_a1a152ae17, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7e860cc724a1a42b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def extended_gcd(a, b):
    if a == 0:
        return (b, 0, 1)
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return (gcd, x, y)

def mod_inverse(a, m):
    gcd, x, _ = extended_gcd(a, m)
    if gcd != 1:
        raise ValueError("Modular inverse does not exist")
    else:
        return x % m

def gaussian_elimination(A, b, p):
    n = len(A)
    for i in range(n):
        max_row = i
        for k in range(i + 1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        factor = (A[i][i] * mod_inverse(A[i][i], p)) % p
        for j in range(i + 1, n):
            A[j][i] = (A[j][i] * factor) % p
            b[j] = (b[j] * factor) % p
    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) * mod_inverse(A[i][i], p) % p
    return x

def tropical_rank(matrix):
    n = len(matrix)
    A = [[0 if i == j else float('inf') for j in range(n)] for i in range(n)]
    b = [float('-inf')] * n
    for i in range(n):
        for j in range(n):
            if matrix[i][j] != 0:
                A[i][j] = matrix[i][j]
```

```

        b[j] = max(b[j], matrix[i][j])
    x = gaussian_elimination(A, b, 2)
    return sum(1 for xi in x if xi != float('-inf'))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    A = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]
    B = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]
    r1 = tropical_rank(A)
    r2 = tropical_rank(B)
    C = [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
    r3 = tropical_rank(C)
    instances_tested = 1
    conjecture_holds = r3 <= r1 + r2
    counterexample = "" if conjecture_holds else "r3 > r1 + r2"
    return {
        "metric_name": "tropical proof rank",
        "metric_value": r3,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i + 3 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"r3 > r1 + r2\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e5df8221.py", line 88, in <module>
    result = run_trial(seed)

```

```

^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e5df8221.py", line 68, in run_trial
    r1 = tropical_rank(A)
    ^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e5df8221.py", line 60, in tropical_rank
    x = gaussian_elimination(A, b, 2)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e5df8221.py", line 42, in gaussian_elim
    factor = (A[i][i] * mod_inverse(A[i][i], p)) % p
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e5df8221.py", line 29, in mod_inverse
    raise ValueError("Modular inverse does not exist")
ValueError: Modular inverse does not exist

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to modular inverse error, preventing validation of conjecture | next:
Check if modulus is prime and verify matrix entries' coprimality with modulus

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	ollama_remote	qwen3:8b	0	0	24241
2	novelty	ollama_remote	qwen3:8b	0	0	20771
3	novelty	ollama_remote	qwen3:8b	0	0	17576
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17243
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12018
6	judge	ollama_remote	qwen3:8b	0	0	19351

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 111200 ms total latency. Provider mix: {'ollama_remote': 6}

(full prompt+response transcripts available in `research/audit/3b94aaddf6cd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3b94aaddf6cd.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3b94aaddf6cd.tar.gz` (if generated)